

Лабораторная работа № 7 : Применение режима однократного гаммирования (One-Time Pad)

Эйвази Мани

7.октябрь.2026

Содержание

1	Отчёт по лабораторной работе № 7	2
2	Цели и задачи работы	2
2.1	Цель лабораторной работы	2
2.2	Задачи	2
3	Процесс выполнения лабораторной работы	3
3.1	Создание программы <code>otp.py</code>	3
3.2	Запуск программы	4
3.3	Подбор ключа для сообщения «С Новым Годом, друзья!» (пункт 4)	5
3.4	Проверка расшифрования с подобранным ключом (пункт 2)	6
4	Ответы на контрольные вопросы	7
5	Выводы по проделанной работе	8
5.1	Выводы	8
6	Цель работы достигнута:	9

1 Отчёт по лабораторной работе № 7

Применение режима одноразового гаммирования (One-Time Pad)

2 Цели и задачи работы

2.1 Цель лабораторной работы

Освоить на практике применение режима одноразового гаммирования.

2.2 Задачи

1. Требуется разработать приложение, позволяющее шифровать и дешифровать данные в режиме одноразового гаммирования. Приложение должно:
2. Определить вид шифротекста при известном ключе и известном открытом тексте.
3. Определить ключ, с помощью которого шифротекст может быть преобразован в некоторый фрагмент текста, представляющий собой один из возможных вариантов прочтения открытого текста.
4. Подобрать ключ, чтобы получить сообщение «С Новым Годом, друзья!».

3 Процесс выполнения лабораторной работы

3.1 Создание программы otp.py

Действие: В домашней директории /lab_07 создать файл otp.py с реализацией режима однократного гаммирования.

```
cd ~/lab_07 cat > otp.py
```

Листинг программы otp.py:

```
root@localhost:~/lab_07# cat crypto.txt
#!/usr/bin/env python3
"""
One-Time Pad (Gamma) Encryption Lab
"""

def xor_bytes(data: bytes, key: bytes) -> bytes:
    """XOR two byte sequences of equal length."""
    if len(data) != len(key):
        raise ValueError("Data and key must be the same length")
    return bytes(a ^ b for a, b in zip(data, key))

def hex_to_bytes(hex_str: str) -> bytes:
    return bytes.fromhex(hex_str.replace(" ", ""))

def bytes_to_hex(b: bytes) -> str:
    return " ".join(f"{x:02X}" for x in b)

def str_to_bytes(s: str, encoding='cp1251') -> bytes:
    """Convert string to bytes. CP1251 for Cyrillic text."""
    return s.encode(encoding)

def bytes_to_str(b: bytes, encoding='cp1251') -> str:
    return b.decode(encoding, errors='replace')

def main():
    print("=" * 50)
    print("ONE-TIME PAD (GAMMA) ENCRYPTION LAB")
    print("=" * 50)
    print("1. Encrypt (key + plaintext -> ciphertext)")
    print("2. Decrypt (key + ciphertext -> plaintext)")
    print("3. Recover key (plaintext + ciphertext -> key)")
    print("4. Find key for target message")
    print("5. Run example from lab")
    print("=" * 50)

    choice = input("Choose option (1-5): ").strip()

    if choice == "1":
        key_hex = input("Enter key (hex): ")
        plain_hex = input("Enter plaintext (hex): ")
        key = hex_to_bytes(key_hex)
        plain = hex_to_bytes(plain_hex)
        cipher = xor_bytes(plain, key)
        print("Ciphertext (hex):", bytes_to_hex(cipher))
```

```

    if choice == "1":
        key_hex = input("Enter key (hex): ")
        plain_hex = input("Enter plaintext (hex): ")
        key = hex_to_bytes(key_hex)
        plain = hex_to_bytes(plain_hex)
        cipher = xor_bytes(plain, key)
        print("Ciphertext (hex):", bytes_to_hex(cipher))

    elif choice == "2":
        key_hex = input("Enter key (hex): ")
        cipher_hex = input("Enter ciphertext (hex): ")
        key = hex_to_bytes(key_hex)
        cipher = hex_to_bytes(cipher_hex)
        plain = xor_bytes(cipher, key)
        print("Plaintext (hex):", bytes_to_hex(plain))
        print("Plaintext (text):", bytes_to_str(plain))

    elif choice == "3":
        plain_hex = input("Enter plaintext (hex): ")
        cipher_hex = input("Enter ciphertext (hex): ")
        plain = hex_to_bytes(plain_hex)
        cipher = hex_to_bytes(cipher_hex)
        key = xor_bytes(cipher, plain)
        print("Recovered key (hex):", bytes_to_hex(key))

    elif choice == "4":
        cipher_hex = input("Enter ciphertext (hex): ")
        target = input("Enter target plaintext: ")
        cipher = hex_to_bytes(cipher_hex)
        target_bytes = str_to_bytes(target)
        if len(target_bytes) != len(cipher):
            print(f"Length mismatch: cipher={len(cipher)} by {len(target_bytes)}")
            print("Adjust your target text to match ciphertext length")
            return
        key = xor_bytes(cipher, target_bytes)
        print("Key to get target message:", bytes_to_hex(key))

    elif choice == "5":
        key_hex = "05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 57"
        plain_hex = "D8 F2 E8 F0 E8 E8 F6 20 2D 20 C2 FB 20 0A"

        key = hex_to_bytes(key_hex)
        plain = hex_to_bytes(plain_hex)

        print("\n--- Example from Lab ---")
```

```

cipher = hex_to_bytes(cipher_hex)
target_bytes = str_to_bytes(target)
if len(target_bytes) != len(cipher):
    print(f"Length mismatch: cipher={len(cipher)} bytes, target={len(target_bytes)} bytes")
    print("Adjust your target text to match ciphertext length.")
    return
key = xor_bytes(cipher, target_bytes)
print("Key to get target message:", bytes_to_hex(key))

elif choice == "5":

    key_hex = "05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 57 FF C8 0B B2 70 54"
    plain_hex = "D8 F2 E8 F0 EB E8 F6 20 2D 20 C2 FB 20 C3 E5 F0 EE E9 21 21"

    key = hex_to_bytes(key_hex)
    plain = hex_to_bytes(plain_hex)

    print("\n--- Example from Lab ---")
    print("Key:", key_hex)
    print("Plaintext (hex):", plain_hex)
    print("Plaintext (text):", bytes_to_str(plain))

    cipher = xor_bytes(plain, key)
    print("Ciphertext (hex):", bytes_to_hex(cipher))

    recovered_key = xor_bytes(cipher, plain)
    print("Recovered key (hex):", bytes_to_hex(recovered_key))
    print("Key match:", bytes_to_hex(recovered_key) == key_hex)

    wrong_key_hex = "05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 55 F4 D3 07 BB BC 54"
    wrong_key = hex_to_bytes(wrong_key_hex)
    wrong_plain = xor_bytes(cipher, wrong_key)
    print("\n--- Wrong Key Decryption ---")
    print("Wrong key:", wrong_key_hex)
    print("Result (hex):", bytes_to_hex(wrong_plain))
    print("Result (text):", bytes_to_str(wrong_plain))

else:
    print("Invalid choice")

if __name__ == "__main__":
    main()

```

Рис. 2,3,4 — python

3.2 Запуск программы

Действие: Сделать файл исполняемым и запустить. Выполнение примера из лабораторной (пункт 5)

```

root@localhost:~/lab_07# chmod +x otp.py
root@localhost:~/lab_07# ./otp.py
=====
ONE-TIME PAD (GAMMA) ENCRYPTION LAB
=====
1. Encrypt (key + plaintext -> ciphertext)
2. Decrypt (key + ciphertext -> plaintext)
3. Recover key (plaintext + ciphertext -> key)
4. Find key for target message
5. Run example from lab
=====
Choose option (1-5): 5

--- Example from Lab ---
Key: 05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 57 FF C8 0B B2 70 54
Plaintext (hex): D8 F2 E8 F0 EB E8 F6 20 2D 20 C2 FB 20 C3 E5 F0 EE E9 21 21
Plaintext (text): Штирлиц - Вы Герой!!
Ciphertext (hex): DD FE FF 8F E5 A6 C1 F2 B9 30 CB D5 02 94 1A 38 E5 5B 51 75
Recovered key (hex): 05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 57 FF C8 0B B2 70 54
Key match: True

--- Wrong Key Decryption ---
Wrong key: 05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 55 F4 D3 07 BB BC 54
Result (hex): D8 F2 E8 F0 EB E8 F6 20 2D 20 C2 FB 20 C1 EE EB E2 E0 ED 21
Result (text): Штирлиц - Вы Болван!

```

Рис. 5 - Отображается меню программы.

При правильном ключе получается исходный текст «Штирлиц – Вы Герой!!». При неправильном ключе получается другой осмысленный текст «Штирлиц – Вы Болван!». Это демонстрирует, что при однократном гаммировании даже неверный ключ даёт осмысленный текст, что затрудняет криптоанализ.

3.3 Подбор ключа для сообщения «С Новым Годом, друзья!» (пункт 4)

Действие: Выбрать пункт 4 — «Find key for target message».

```

root@localhost:~/lab_07# cat cypher
:ipthertext: DD FE FF 8F E5 A6 C1 F2 B9 30 CB D5 02 94 1A 38 E5 5B 51 75
target plaintext: С Новым Годом, друзья
root@localhost:~/lab_07# ./otp.py
=====
ONE-TIME PAD (GAMMA) ENCRYPTION LAB
=====
1. Encrypt (key + plaintext -> ciphertext)
2. Decrypt (key + ciphertext -> plaintext)
3. Recover key (plaintext + ciphertext -> key)
4. Find key for target message
5. Run example from lab
=====
Choose option (1-5): 4
Enter ciphertext (hex): DD FE FF 8F E5 A6 C1 F2 B9 30 CB D5 02 94 1A 38 E5 5B 51 75
Enter target plaintext: С Новым Годом, друзья
key to get target message: 0C DE 32 61 07 5D 2D D2 7A DE 2F 3B EE B8 3A DC 15 A8 B6 8A

```

Рис. 6 — Ключ вычислен по формуле $K_i = C_i \oplus P_i$, где C_i — шифротекст, P_i — целевой открытый текст.

3.4 Проверка расшифрования с подобранным ключом (пункт 2)

Действие: Выбрать пункт 2 — «Decrypt».

```
root@localhost:~/lab_07# ./otp.py
=====
ONE-TIME PAD (GAMMA) ENCRYPTION LAB
=====
1. Encrypt (key + plaintext -> ciphertext)
2. Decrypt (key + ciphertext -> plaintext)
3. Recover key (plaintext + ciphertext -> key)
4. Find key for target message
5. Run example from lab
=====
Choose option (1-5): 2
Enter key (hex): 0C DE 32 61 07 5D 2D D2 7A DE 2F 3B EE B8 3A DC 15 A8 B6 8A
Enter ciphertext (hex): DD FE FF BF E5 A6 C1 F2 B9 30 CB D5 02 94 1A 38 E5 5B 51 75
Plaintext (hex): D1 20 CD EE E2 FB EC 20 C3 EE E4 EE EC 2C 20 E4 F0 F3 E7 FF
Plaintext (text): С Новым Годом, друзья
```

Рис. 7 — С подобранным ключом шифротекст успешно расшифрован в целевое сообщение «С Новым Годом, друзья».

4 Ответы на контрольные вопросы

1. Поясните смысл одноратного гаммирования. Это метод симметричного шифрования, при котором на открытый текст накладывается случайная последовательность бит (гамма) той же длины, что и текст. Наложение выполняется операцией XOR. Расшифрование выполняется той же операцией с тем же ключом.

2. Перечислите недостатки одноратного гаммирования. Ключ должен быть истинно случайным. Ключ должен быть той же длины, что и сообщение. Ключ можно использовать только один раз. Сложность генерации и распространения ключей большой длины. При компрометации ключа вся переписка раскрывается.

3. Перечислите преимущества одноратного гаммирования. Абсолютная стойкость (доказана К. Шенноном) при выполнении всех условий. Простота реализации (операция XOR). Высокая скорость шифрования и расшифрования. Криптоанализ невозможен при правильном использовании.

4. Почему длина открытого текста должна совпадать с длиной ключа? Потому что гамма накладывается побитово на каждый символ текста. Если ключ короче, его придётся повторять, что нарушает условие одноратности и снижает стойкость.

5. Какая операция используется в режиме одноратного гаммирования, назовите её особенности?

- Используется операция XOR (сложение по модулю 2).
- Особенности: $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$. Двойное применение XOR с одним и тем же ключом восстанавливает исходные данные.

6. Как по открытому тексту и ключу получить шифротекст?

- $C_i = P_i \oplus K_i$, где C_i — шифротекст, P_i — открытый текст, K_i — ключ.

7. Как по открытому тексту и шифротексту получить ключ?

- $C_i = P_i \oplus K_i$, где C_i — шифротекст, P_i — открытый текст, K_i — ключ.

8. В чём заключаются необходимые и достаточные условия абсолютной стойкости шифра? Полная случайность ключа. Равенство длин ключа и открытого текста. Одноратное использование ключа.

5. Содержание отчёта

- ☐ Титульный лист
- ☐ Цель работы
- ☐ Описание процесса выполнения с командами и снимками экрана
- ☐ Листинг программы `otr.py`
- ☐ Результаты работы программы
- ☐ Ответы на контрольные вопросы
- ☐ Выводы, согласованные с целью работы

5 Выводы по проделанной работе

5.1 Выводы

В ходе лабораторной работы я:

- Разработал приложение `otr.py` на языке Python, реализующее режим однократного гаммирования.
- Реализовал функции:
- Шифрование (ключ + открытый текст → шифротекст).
- Расшифрование (ключ + шифротекст → открытый текст).
- Восстановление ключа (открытый текст + шифротекст → ключ).
- Подбор ключа для целевого сообщения.
- Выполнил пример из лабораторной работы:
- Зашифровал сообщение «Штирлиц – Вы Герой!!» ключом Центра.
- Получил шифротекст DD FE FF 8F E5 A6 C1 F2 B9 30 CB D5 02 94 1A 38 E5 5B 51 75.
- При расшифровании неправильным ключом получил осмысленный текст «Штирлиц – Вы Болван!», что демонстрирует стойкость шифра.
- Подобрал ключ для сообщения «С Новым Годом, друзья!»:
- Ключ: 0C DE 32 61 07 5D 2D D2 7A DE 2F 3B EE B8 3A DC 15 A8 B6 8A.

- Убедился, что с этим ключом шифротекст расшифровывается в целевое сообщение.
- Убедился, что однократное гаммирование обеспечивает абсолютную стойкость при выполнении всех условий (случайный ключ, равенство длин, однократное использование).

6 Цель работы достигнута:

освоено практическое применение режима однократного гаммирования.

7. Краткая сводка команд

bash

Переход в рабочую директорию

cd ~/lab_07

Создание программы

cat > otp.py

(вставить листинг программы)

Сделать исполняемым и запустить

chmod +x otp.py

./otp.py

Пример работы:

Пункт 5 — пример из лабораторной

Пункт 4 — подбор ключа для целевого сообщения

Пункт 2 — проверка расшифрования

Просмотр файла cipher

cat cipher